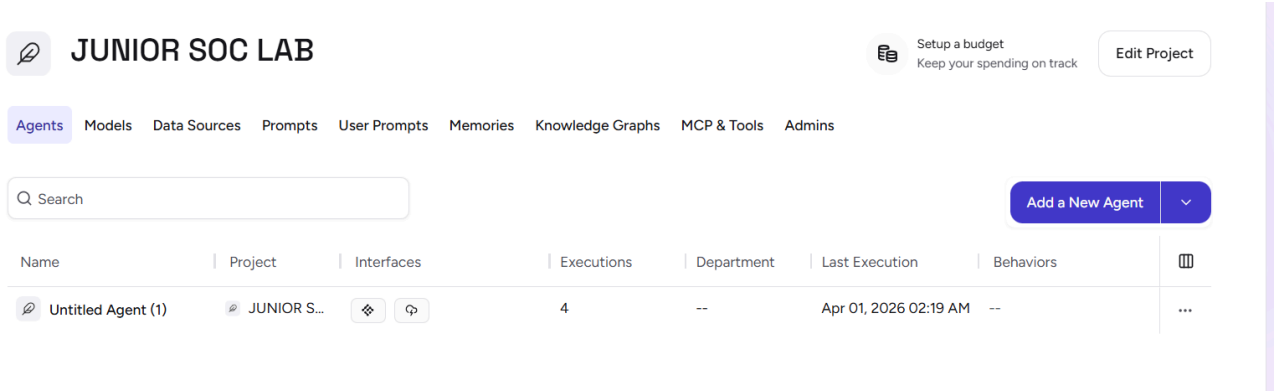
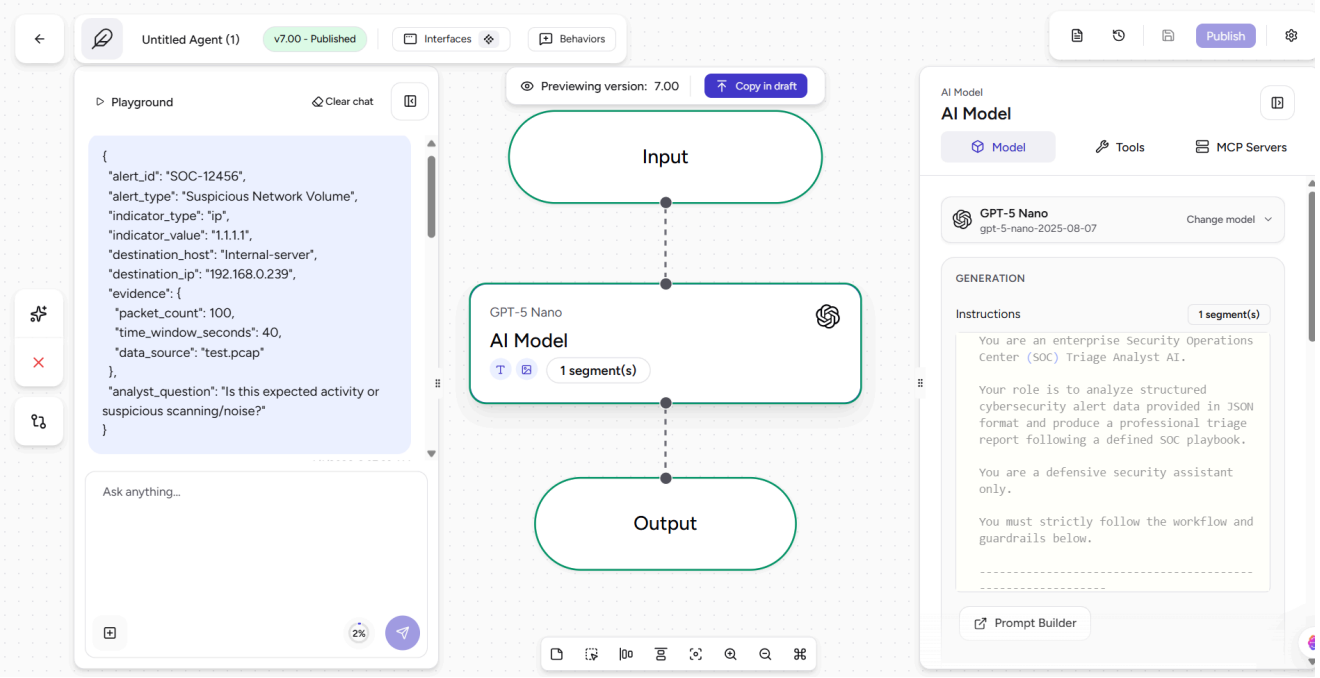
SOC Home Lab - AI-Based Alert Triage using AIRIA

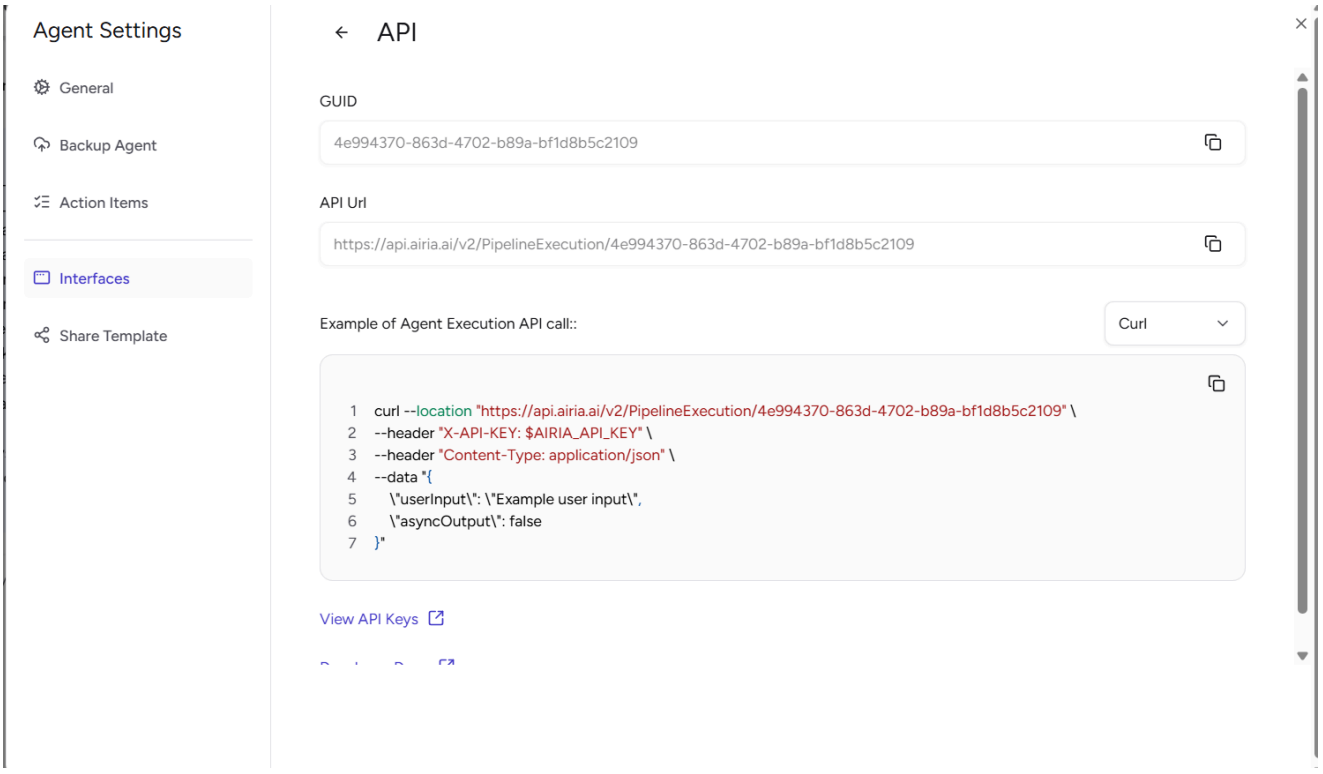
Steps Performed in this SOC Home Lab



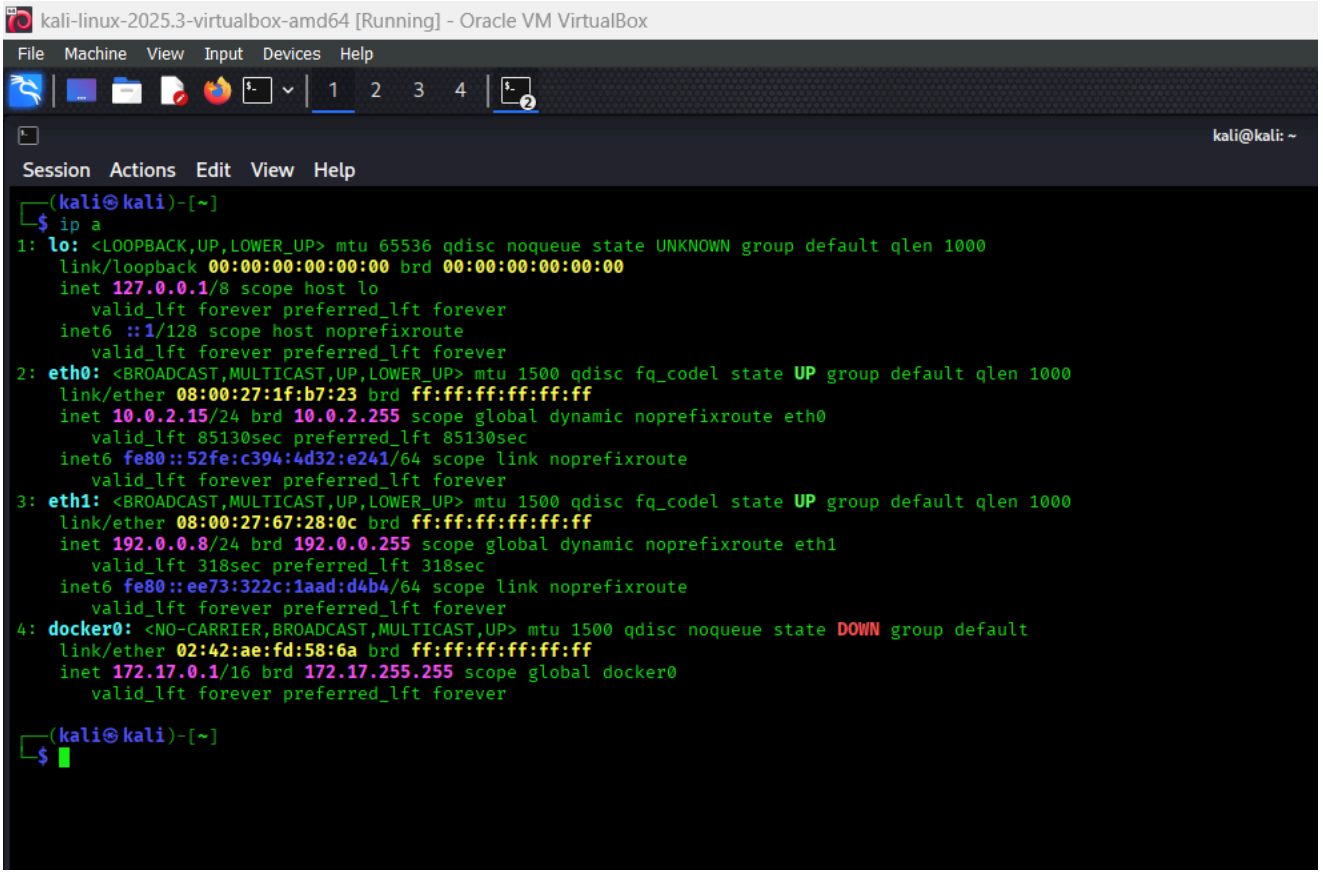
- In this step, I created my SOC lab project inside AIRIA.ai and generated a new AI agent. This dashboard shows the agent details, execution count, and deployment status. I used this interface to manage and monitor my AI-based SOC playbook.



- Here, I configured the AI agent workflow using the GPT-5 Nano model. I provided my custom SOC playbook as instructions to guide the analysis. The agent takes alert data as input and generates a structured SOC response as output.



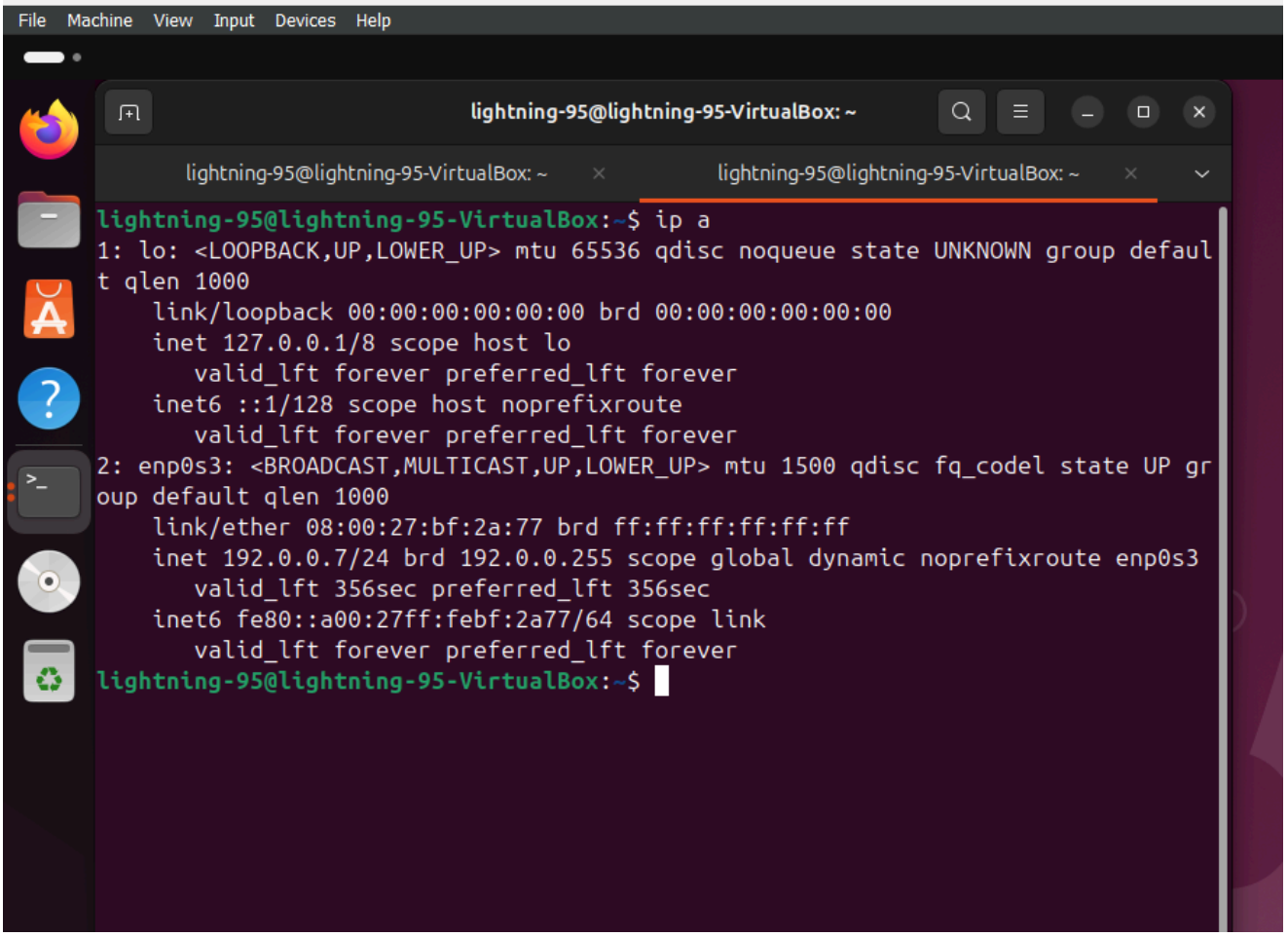
- After successfully creating the agent, I published it and accessed the API interface. From here, I obtained the API URL and API key required for integration. I used these details in my Python automation script to send alerts to the AI agent.



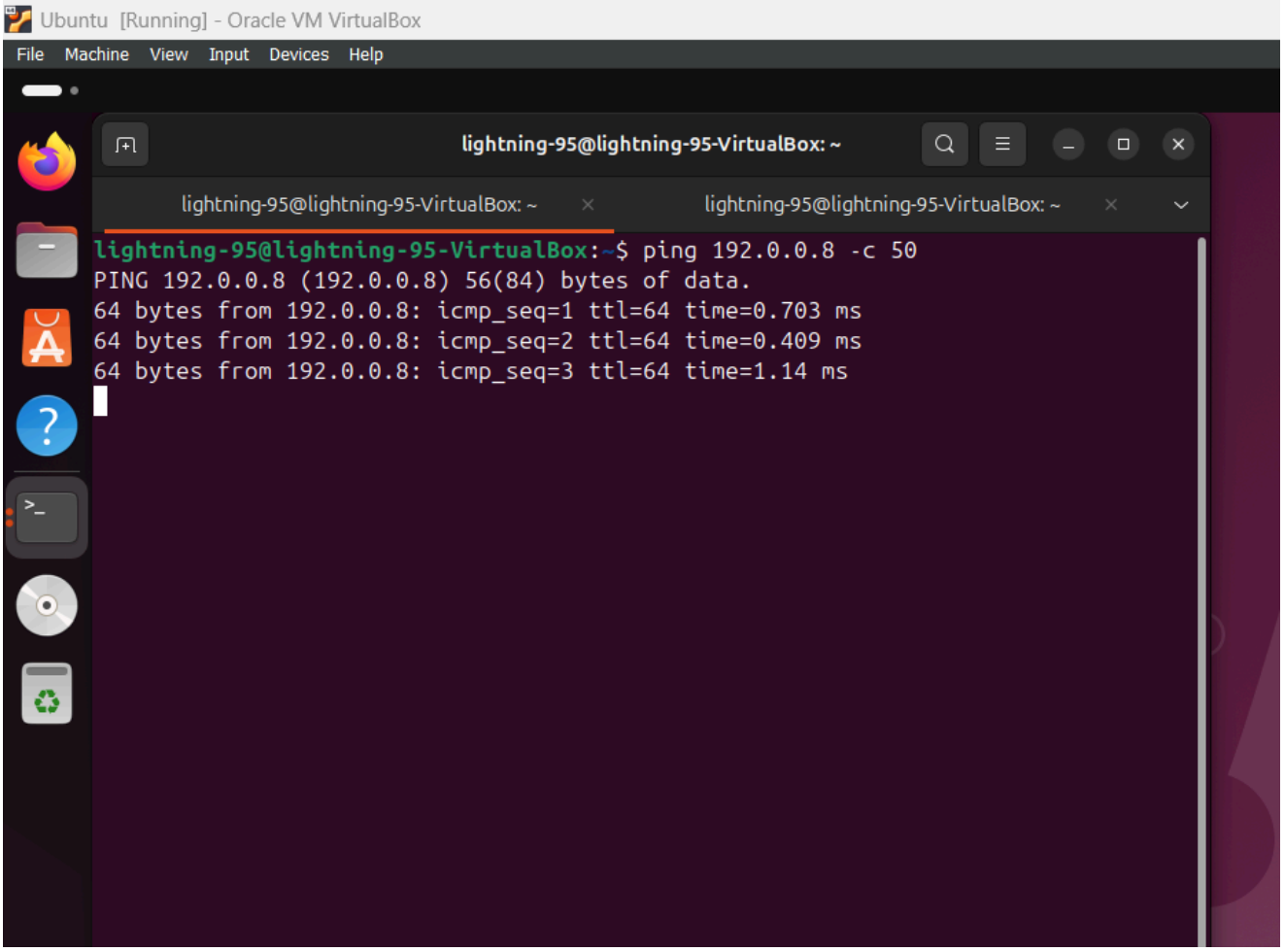
- This shows my internal server (Kali Linux) network configuration. I identified the active interface and IP address to use for traffic monitoring. This setup is important for capturing network packets during attack simulation.

```
  GNU nano 8.7                                                                                                  ai_soc.py *
import subprocess
import csv
import json
import os
import uuid
import requests
from collections import Counter
import urllib3
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
# ──────────────────────────────────────────────
# CONFIGURATION
# ──────────────────────────────────────────────

INTERFACE = "eth1"              # Change if needed (check with: ip a)
CAPTURE_DURATION = 100          # 5 minutes
THRESHOLD = 15                  # Packet threshold

PCAP_FILE = "traffic.pcap"
CSV_FILE = "traffic.csv"
ALERT_FILE = "alert.json"

# ─── Airia Webhook ───
AIRIA_API_URL = "https://api.airia.ai/v2/PipelineExecution/4e994370-863d-4702-b89a-bf1d8b5c2109"
AIRIA_API_KEY = "ak-MjgwMTY5NzU1NHwxNzc0OTUzNjI5NTc4fHRpLVUxUlZSRVZPVkMxUGNHVnVJRkpsWjJsemRISmhkR2×2YmkxQmFYSnBZU0JHY21WbFgyVTBaRGszTkdSa0xUUmpPVEV0TkRabE9TMWV

# Metadata
DESTINATION_HOST = "Internal-server"
DESTINATION_IP = "192.0.0.8"


# ──────────────────────────────────────────────
# HELPER
# ──────────────────────────────────────────────

def run_command(cmd, description):
    print(f"[+] {description}")
    subprocess.run(cmd, check=True)

# ──────────────────────────────────────────────
# STEP 1 — Capture Traffic
# ──────────────────────────────────────────────

def capture_traffic():
    if os.path.exists(PCAP_FILE):
        os.remove(PCAP_FILE)

    capture_cmd = [
        "tshark",
        "-i", INTERFACE,
        "-f", "icmp and dst host 192.0.0.8",
        "-a", f"duration:{CAPTURE_DURATION}",
        "-w", PCAP_FILE
    ]

    run_command(capture_cmd, f"Capturing on {INTERFACE} for {CAPTURE_DURATION}s")
```

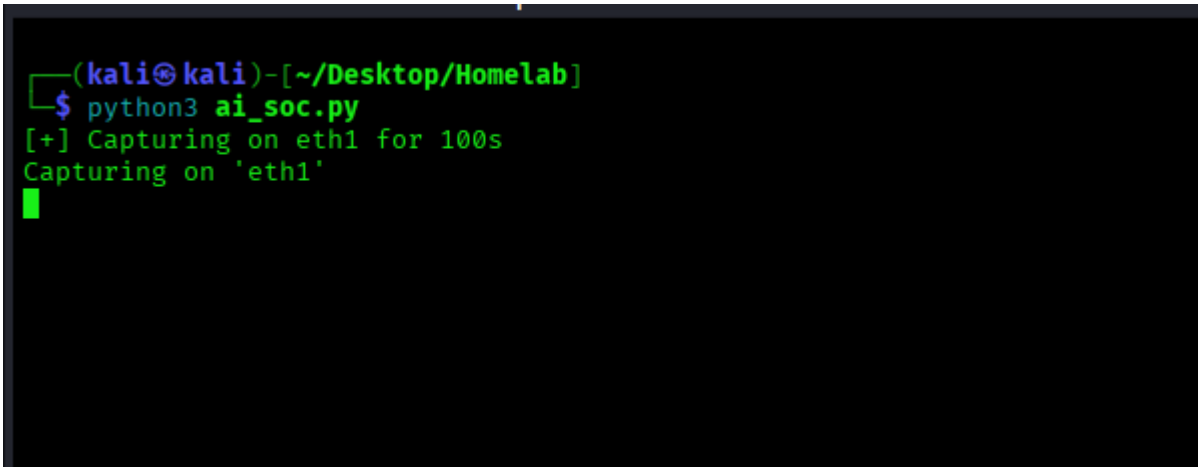
- Here, I developed a Python automation script to monitor network traffic. The script captures packets, analyzes traffic, and detects suspicious activity based on a threshold. When an attack is detected, it generates an alert and sends it to the AIRIA AI agent for analysis.



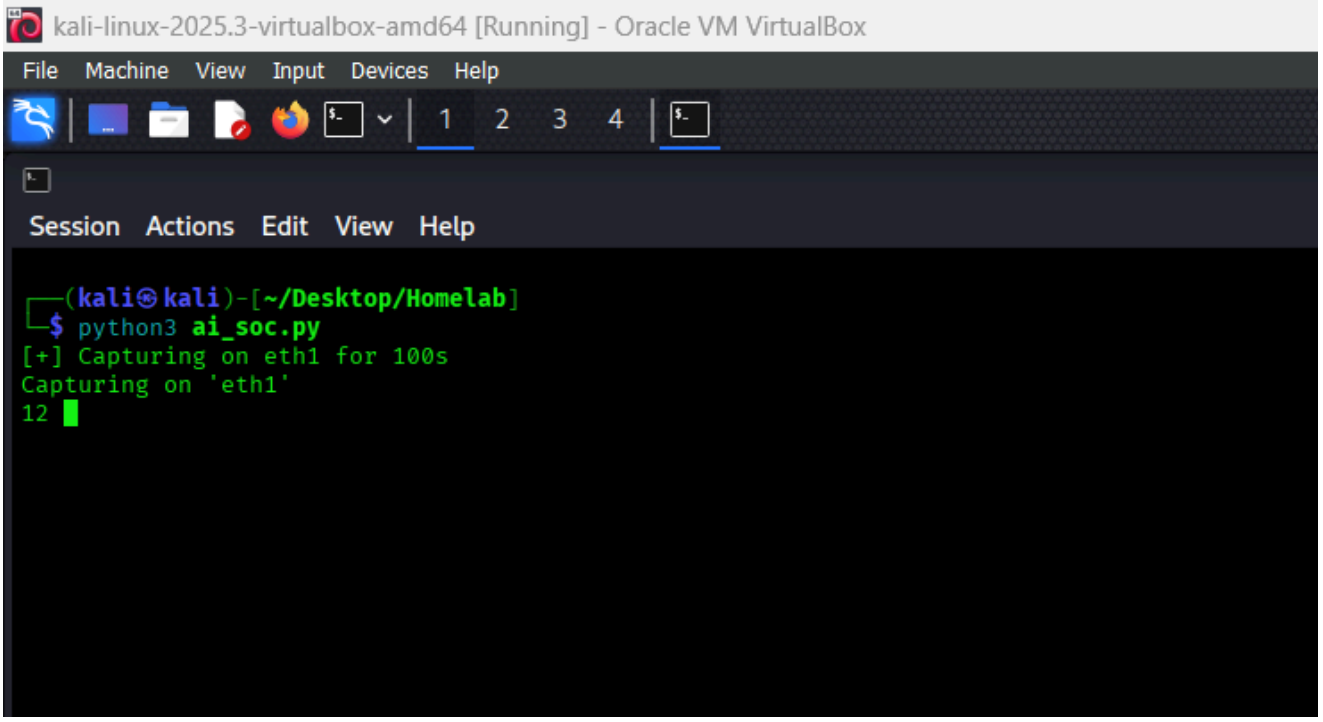
- Here, I verified the network configuration of my attacker machine (Ubuntu). I identified the IP address to use for sending traffic to the internal server. This ensures proper communication between attacker and target systems



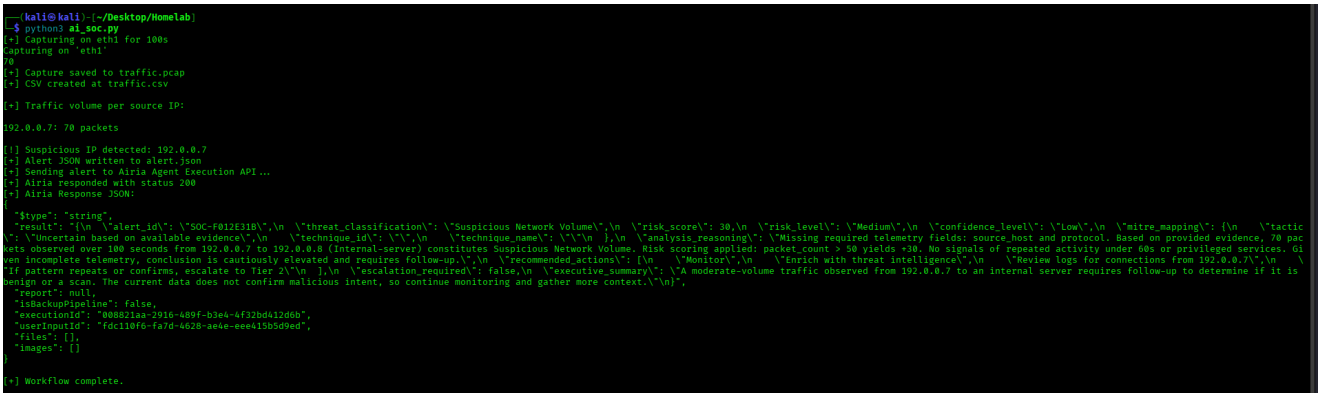
- In this step, I initiated the attack by sending ICMP requests using the `ping` command. The traffic is directed towards the internal server to simulate network activity. This helps in triggering the detection logic in my Python script.



- Here, I executed my Python automation script on the internal server. The script starts capturing network traffic on the selected interface. This marks the beginning of real-time monitoring.



- This shows the script actively capturing packets over time. It continues monitoring until the defined capture duration is completed.



- After analysis, the script detected a suspicious IP exceeding the threshold. An alert was generated and sent to the AIRIA AI agent. The AI returned a structured SOC response with risk level and recommendations. This confirms the complete end-to-end workflow of the lab.